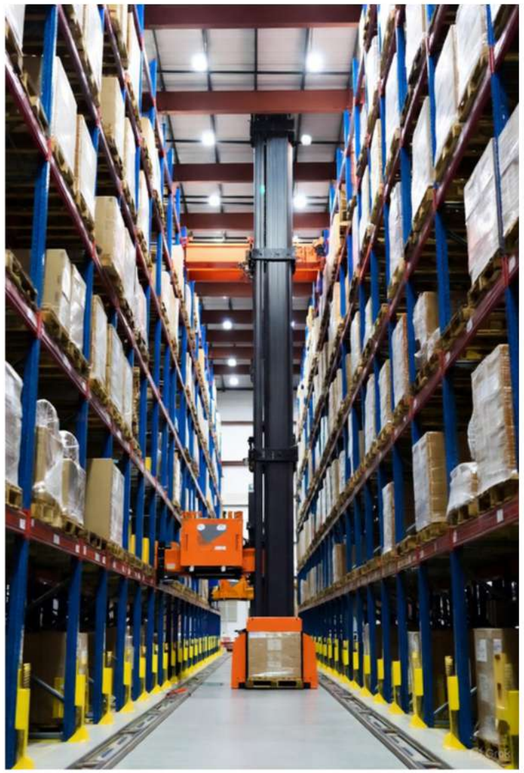
Optimización de Slotting en Almacenes Medición por Tiempo de Viaje Total

Comparación de 2-Opt, Tabu Search y ALNS

Kevin Sneider Vera Galvis

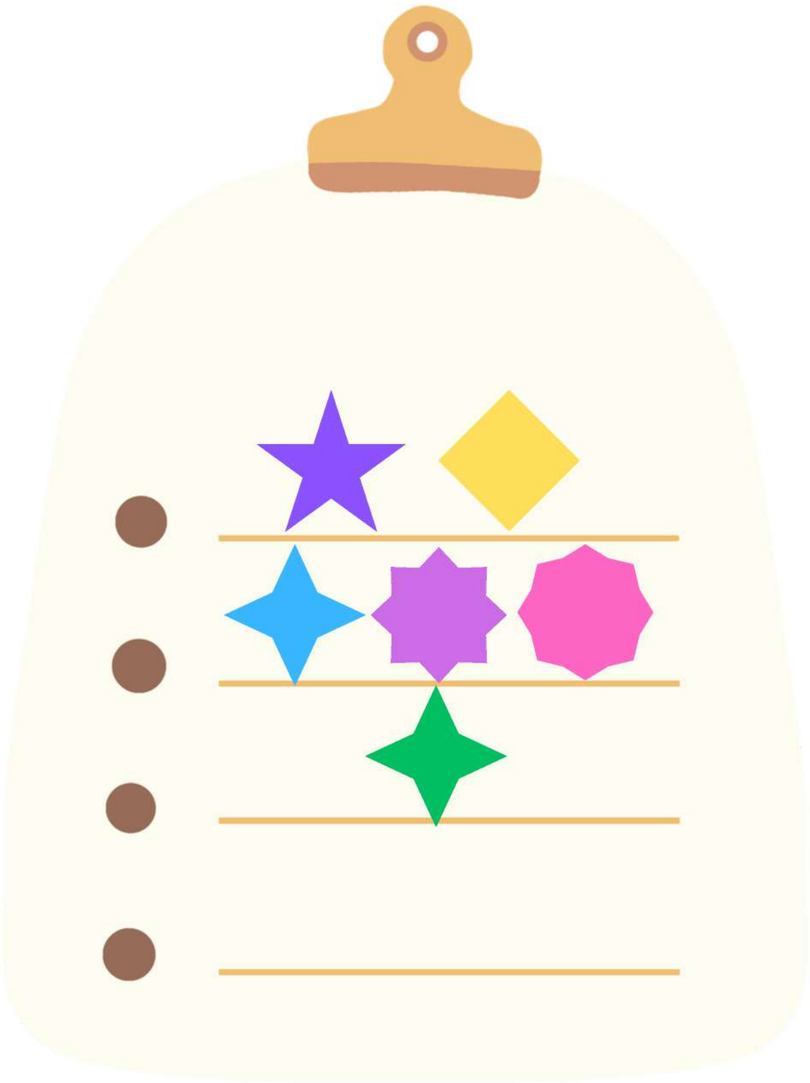
#LaUISqueQueremos

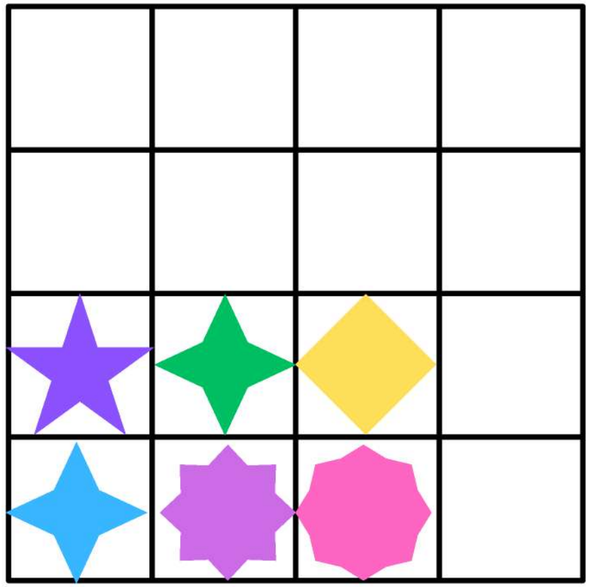
Descripción del Problema



- Estantería: 10 niveles × 30 columnas
- Ocupación: 85%
- 60 pedidos (4-9 ítems cada uno)
- Distancia: Chebyshev (máximo movimiento en X o Y)
- Objetivo: Minimizar Tiempo de Viaje Total (colocación ida-vuelta + recorrido de picking)

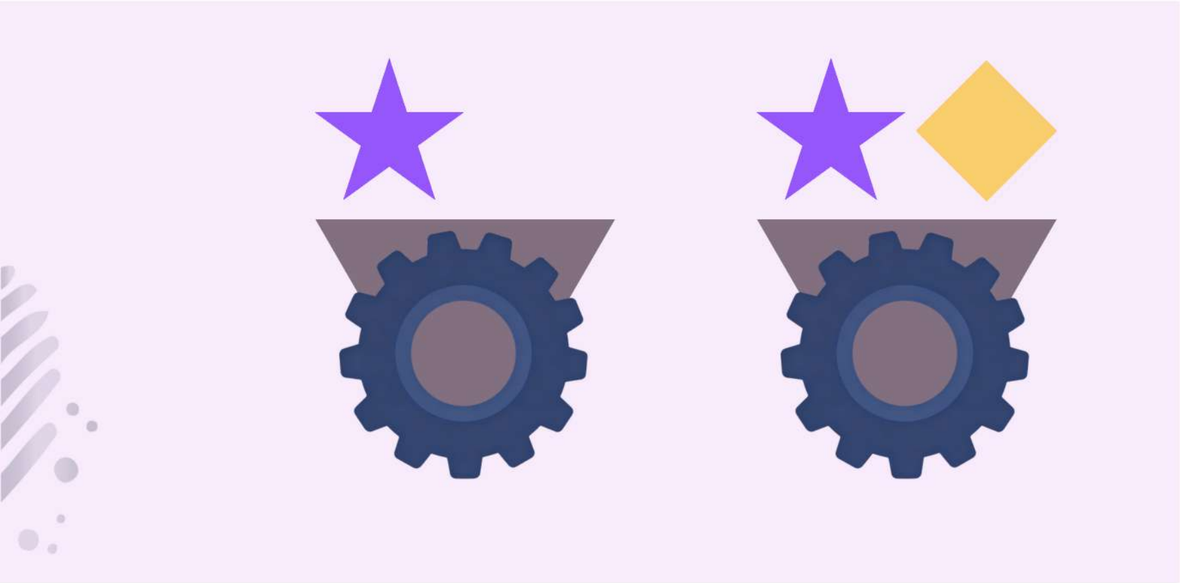
Descripción del Problema

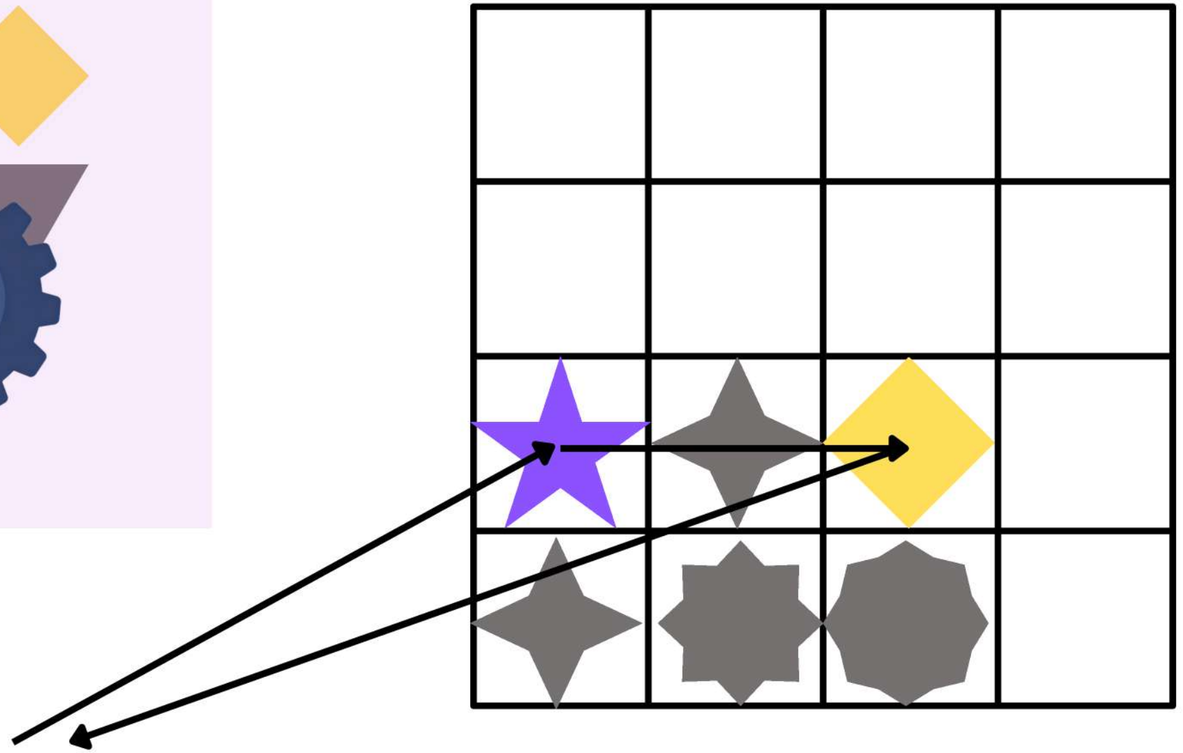




#LaUISqueQueremos

Descripción del Problema





Punto de Entrada
y Salida

#LaUISqueQueremos

Inicialización Greedy (por frecuencia)

Este es el punto de partida que usan todas las heurísticas.

1. Tomar todos los ítems reales que hay que colocar en la estantería.
2. Contar cuántas veces aparece cada ítem en los pedidos (su frecuencia o demanda).
3. Ordenar los ítems de mayor a menor frecuencia (los más pedidos primero).
4. Si dos ítems tienen la misma frecuencia, ordenarlos por su número de identificación (para que sea determinista).
5. Colocar los ítems ordenados en las posiciones disponibles (primero los más frecuentes).
6. Llenar las posiciones sobrantes con ítems dummy (vacíos).
7. Calcular el tiempo total de viaje de esta asignación inicial.
8. Devolver esta asignación como solución de inicio.

Lower Bound (límite inferior teórico – solo colocación)

Este calcula el mejor tiempo posible solo considerando la fase de colocación (ida y vuelta al origen).

1. Contar cuántos ítems reales hay que colocar (n).
2. Crear una tabla (matriz) de costos donde cada fila es un ítem y cada columna es una posición posible.
3. En cada celda de la tabla poner el costo de colocar ese ítem en esa posición: 2 × distancia desde el origen a la posición.
4. Usar el algoritmo de asignación lineal (húngaro) para encontrar la asignación óptima de ítems a posiciones.
5. Sumar los costos de esa asignación óptima.
6. Ese valor es el tiempo mínimo teórico solo para la colocación (ida + vuelta de todos los ítems).
7. Devolver ese valor como lower bound.

Nota: No incluye el tiempo de los tours de picking, por eso es solo un límite parcial.

Algoritmo 2OPt - Pseudocódigo

1. Partir de una solución inicial (greedy).
2. Mientras se pueda mejorar o no se alcance el máximo de iteraciones:

   a. Para cada par de posiciones i y j (donde j > i+1):
   b. Invertir (dar la vuelta) el segmento de la asignación entre i y j. 5. Calcular el nuevo tiempo total de viaje con ese cambio.
   c. Si el tiempo nuevo es mejor que el mejor conocido hasta ahora:
   d. Guardar este cambio como el nuevo mejor. 8. Marcar que hubo mejora.
   e. Si no mejoró, deshacer el cambio (volver al estado anterior).
   f. Si en toda la pasada no hubo ninguna mejora, salir del bucle (se llegó a un óptimo local).
   g. Registrar el mejor tiempo encontrado en esta iteración.

3. Devolver la mejor asignación encontrada y la lista de tiempos por iteración.

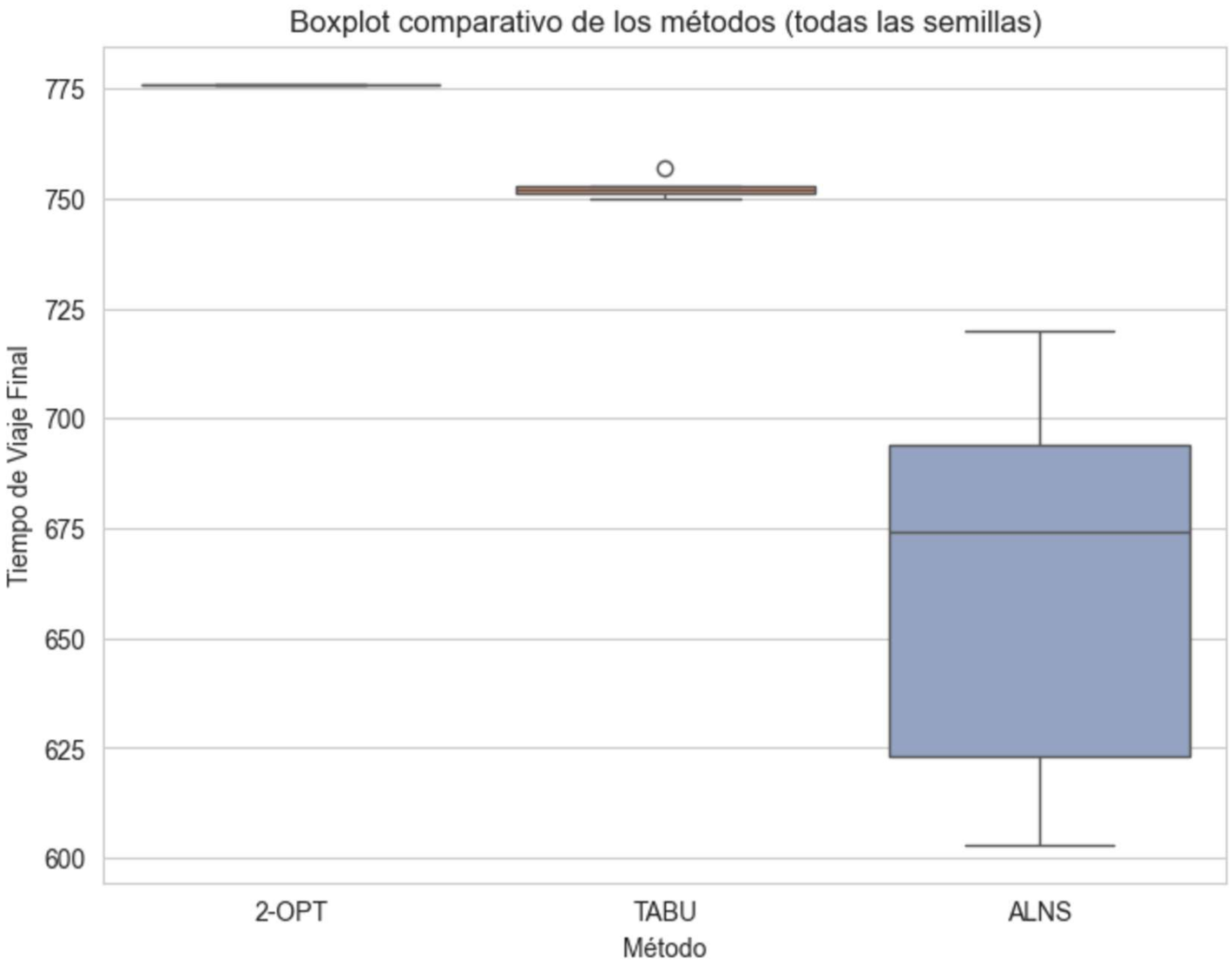
Búsqueda Tabú - Pseudocódigo

1. Partir de una solución inicial (greedy).
2. Crear una lista tabú vacía para recordar movimientos prohibidos temporalmente.
3. Mientras no se llegue al máximo de iteraciones:
4. Generar varios vecinos posibles (intercambiando dos posiciones al azar).
5. Para cada vecino generado:
6. Calcular su tiempo total de viaje.
7. Si el intercambio de posiciones no está prohibido (no aparece en la lista tabú):
8. Guardarlo como candidato válido.
9. Si hay candidatos válidos:
10. Elegir el candidato con el menor tiempo de viaje.
11. Aplicar ese intercambio a la solución actual.
12. Añadir el movimiento realizado a la lista tabú.
13. Si la lista tabú es demasiado larga, eliminar el movimiento más antiguo.
14. Si el nuevo tiempo es mejor que el mejor conocido hasta ahora, actualizar el mejor.
15. Registrar el mejor tiempo encontrado en esta iteración.
16. Devolver la mejor solución y la lista de tiempos por iteracion

ALNS - Pseudocódigo

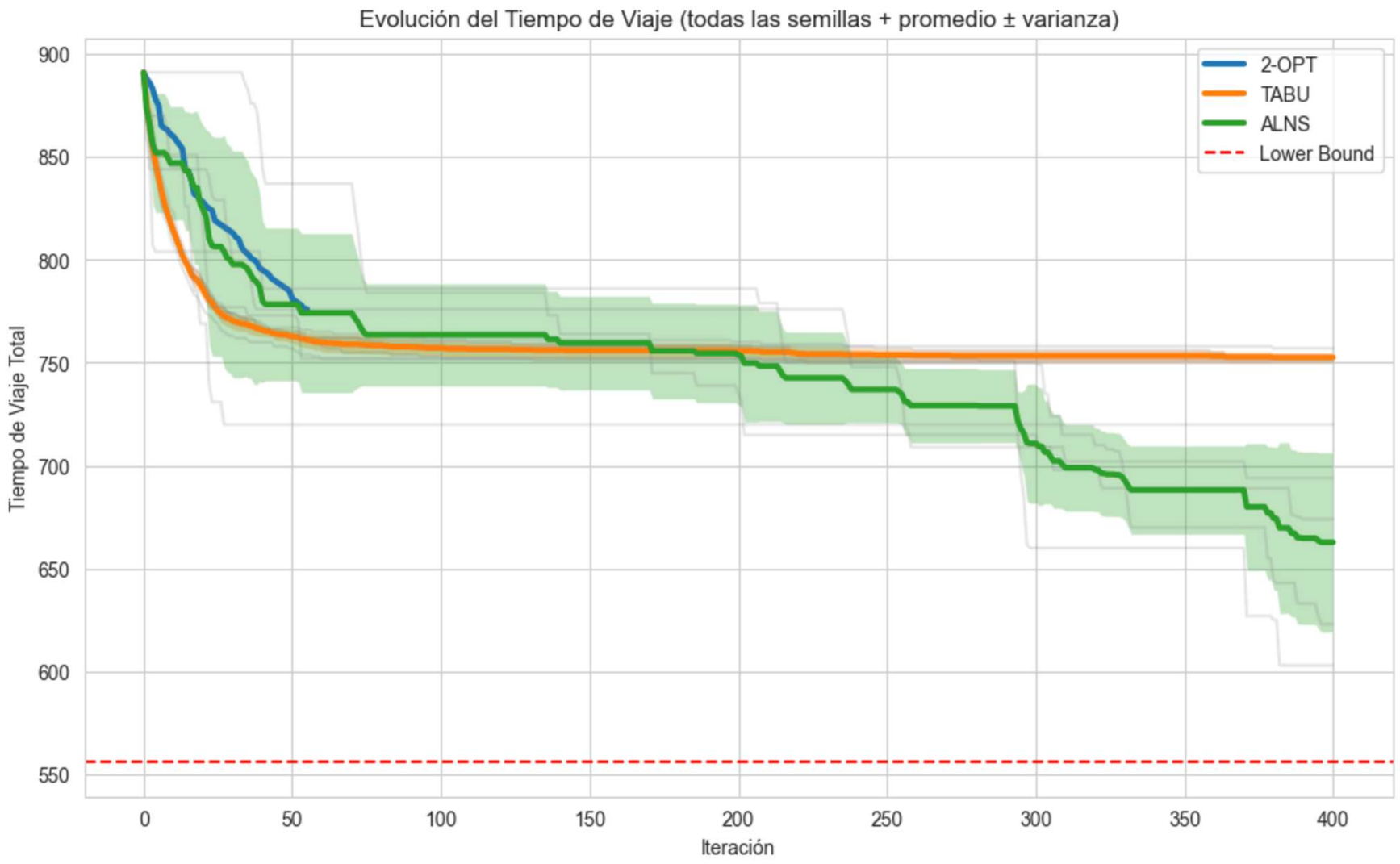
1. Partir de una solución inicial (greedy).
2. Establecer una temperatura inicial alta (para aceptar soluciones peores al principio).
3. Mientras no se llegue al máximo de iteraciones:
4. Hacer una copia de la solución actual.
5. Decidir cuántos ítems destruir (porcentaje destroy_prob).
6. Eliminar aleatoriamente esos ítems de la copia (fase destroy).
7. Ordenar los ítems eliminados por frecuencia descendente.
8. Para cada ítem eliminado:
9. Probar varias posiciones posibles para reinsertarlo (normalmente 15-40 posiciones al azar).
10. Calcular el tiempo de viaje con cada inserción.
11. Elegir la posición que dé el menor tiempo (fase repair greedy).
12. Insertar el ítem en esa mejor posición.
13. Calcular el tiempo total de la nueva solución vecina.
14. Si el tiempo vecino es mejor que el mejor conocido:
15. Actualizar la mejor solución.
16. Si no es mejor, aceptarlo con probabilidad basada en la temperatura.
17. Reducir la temperatura multiplicándola por cooling_rate.
18. Registrar el mejor tiempo encontrado en esta iteración.
19. Devolver la mejor solución y la lista de tiempos por iteración.

Estabilidad de Semilla y Evolución

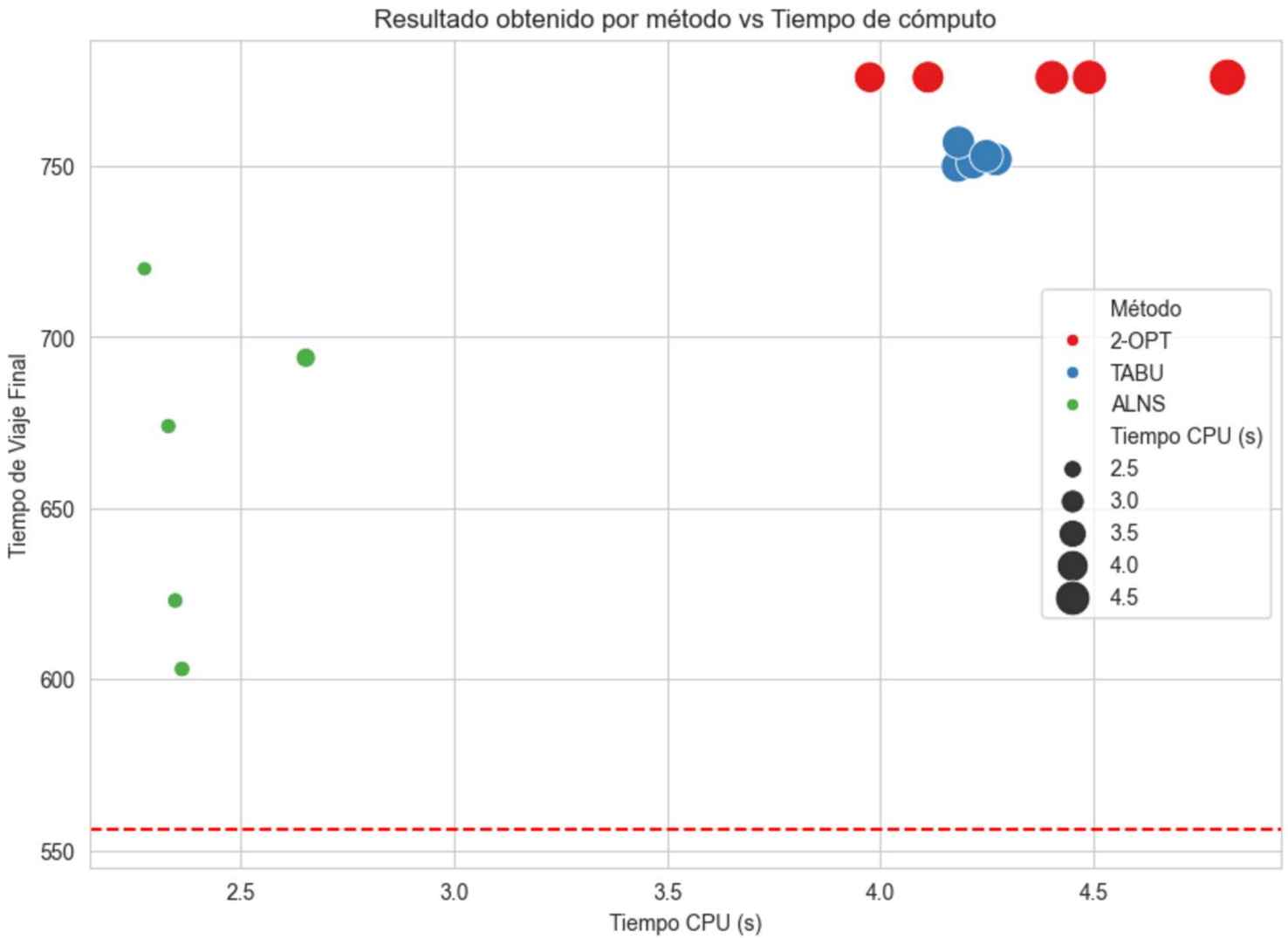


#LaUISqueQueremos

Estabilidad de Semilla y Evolución

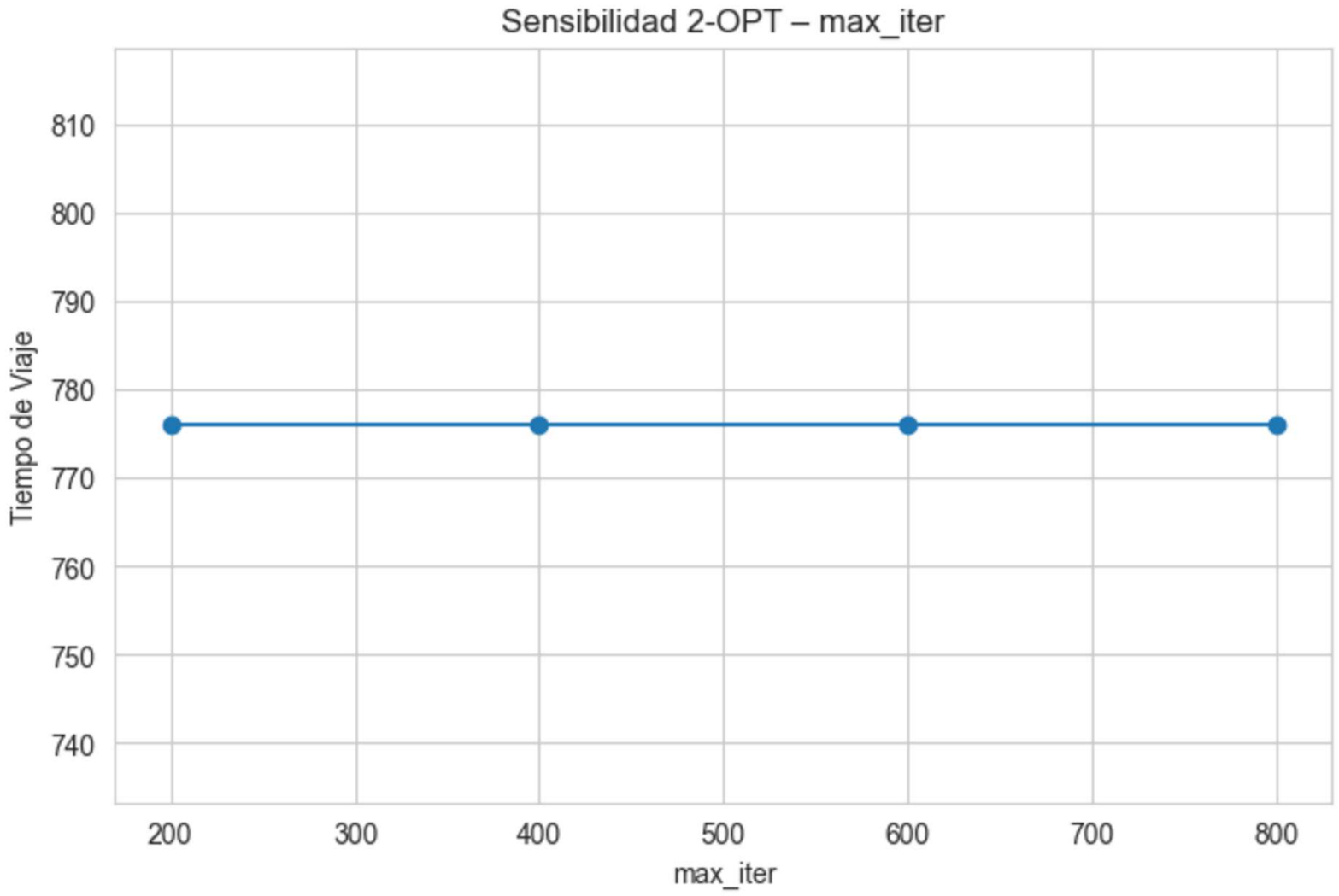


Tiempo de Viaje vs. Tiempo de Computo



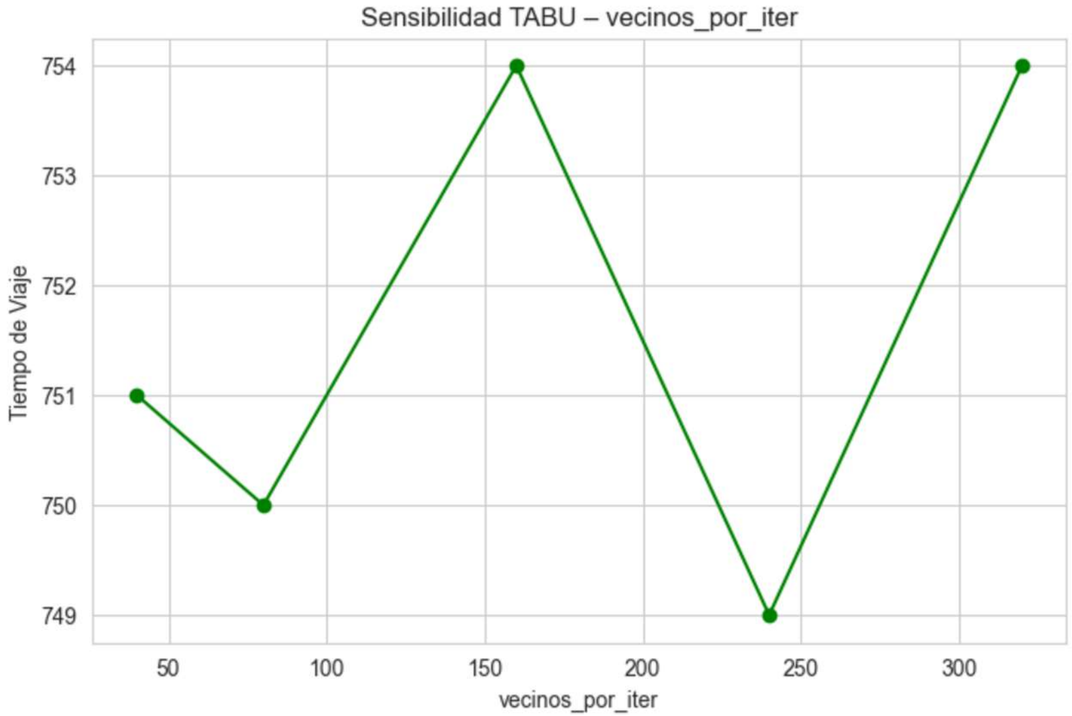
#LaUISqueQueremos

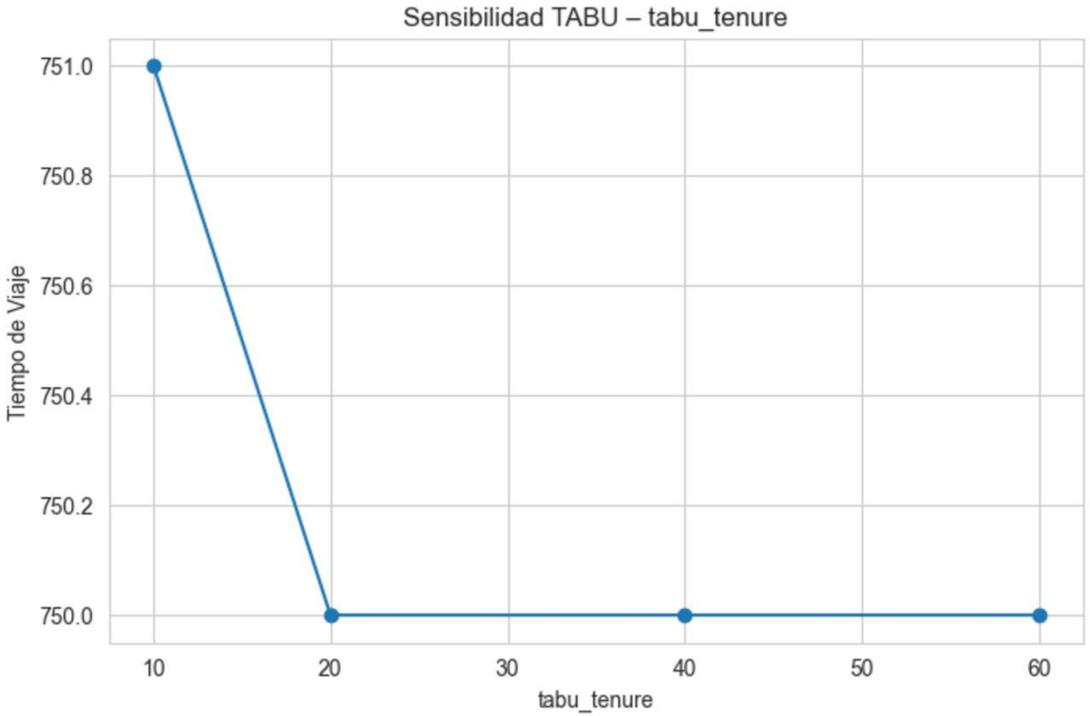
Análisis de Sensibilidad - 2opt



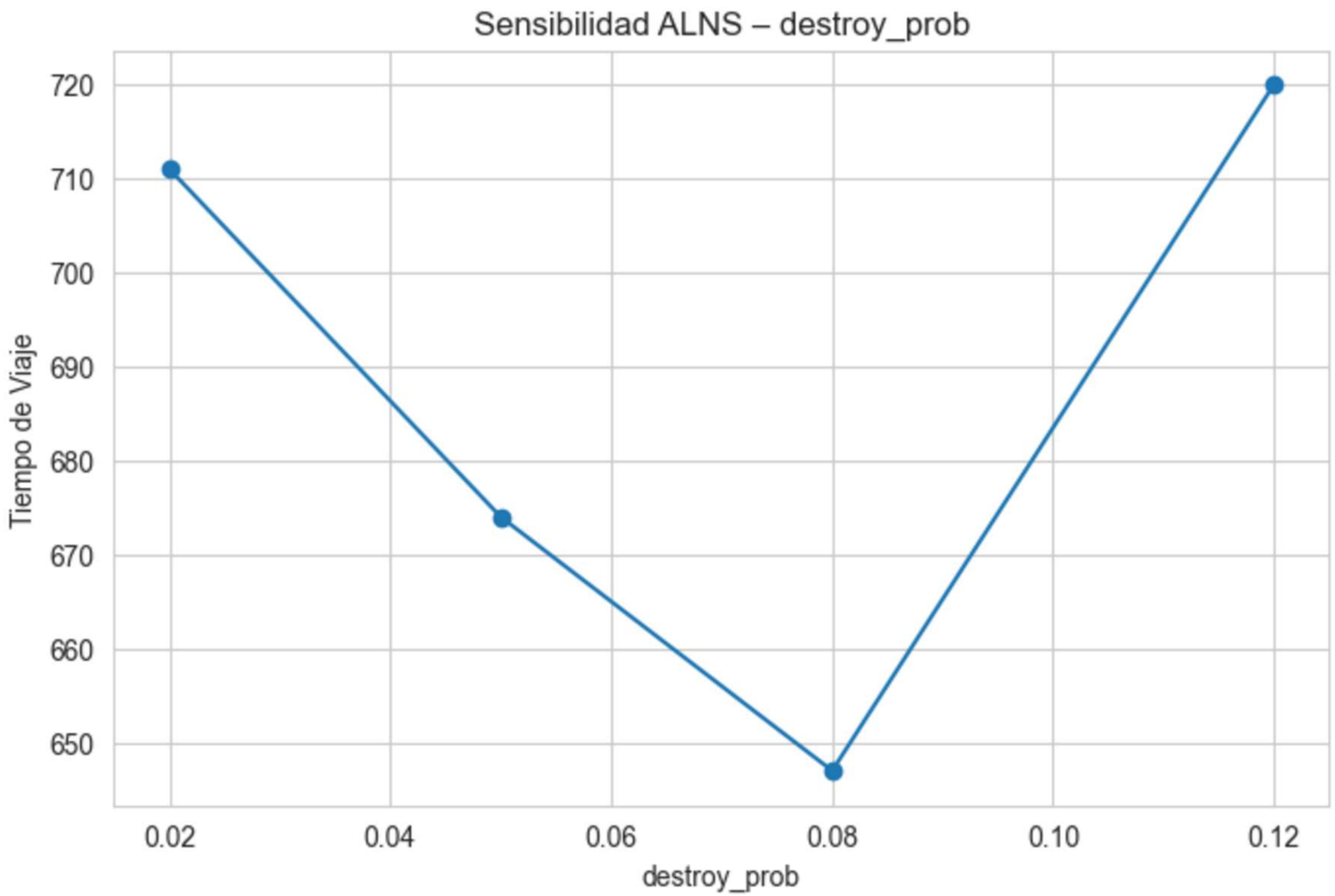
#LaUISqueQueremos

Análisis de Sensibilidad - Búsqueda Tabu





Análisis de Sensibilidad - Búsqueda Tabu



#LaUISqueQueremos

Universidad Industrial de Santander

UIS 1948 2025

¡Gracias!